# C语言字节序：大端与小端详解

深入理解数据在内存中的存储奥秘

## 1. 概念

在计算机内存中或在数字通信链路中，由多个字节组成的数据的字节排列顺序。简单来说，就是当一个数据大于一个字节时（例如，一个int类型通常是4个字节），如何在内存地址中排列这些字节的问题。

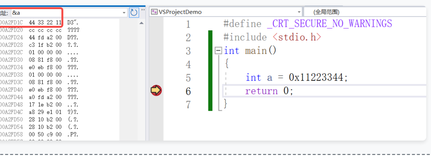
**同时博哥要提醒大家的是：注意是字节序！**

我们来看以下数据在内存当中是怎么存储的？

```c
#include <stdio.h>

int main()
{
    int a = 0x11223344;

    return 0;
}
```




我们可以看到在a中的0x11223344这个数字是按照字节为单位，倒着存储的。这种存储方式就是小端存储！

## 2. 小端字节序

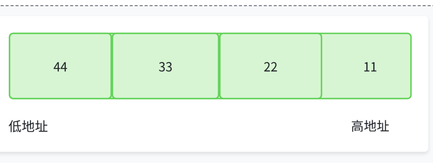
**小端字节序 (Little-Endian)**

是指数据的低位字节内容保存在内存的低地址处，而数据的高位字节内容，保存在内存的高地址处。

0x44这个数据就是低位字节内容，0x11这个数据就是高位字节内容。

这种方式更符合计算机处理数据的方式，因为计算通常从最低位开始，这样可以提高电路处理效率。

**小端字节序**：是大多数现代处理器架构的主流选择，例如Intel的x86和大多数ARM实现。

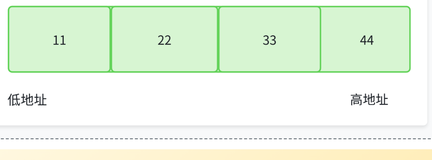



## 3. 大端字节序

**大端字节序 (Big-Endian)**

是指数据的低位字节内容保存在内存的高地址处，而数据的高位字节内容，保存在内存的低地址处。

如果将上述数据存储内存中后如下：




**网络字节序**

**网络协议**：TCP/IP协议栈采用大端字节序，因此大端序也被称为**网络字节序（Network Byte Order）**。可以确保了不同架构的计算机在网络通信时能正确解析数据。当数据在网络上传输时，发送方计算机会将数据从其主机字节序转换成网络字节序（大端），接收方再将其从网络字节序转换回自己的主机字节序。

## 4. 为什么存在两种字节序?

大小端字节序的存在源于不同的计算机体系结构设计，各有优势。

**大端优势：**

- **可读性好**：存储顺序和人类的阅读习惯一致。
- **符号判断快**：只需检查第一个字节就可以判断一个数的正负。

**小端优势：**

- **计算效率高**：计算机电路从低位字节开始处理效率更高。例如，在进行加法或乘法运算时，CPU从低位到高位处理更方便。
- **类型转换方便**：强制类型转换时，无需改变地址即可直接获取低位字节的数据。例如，将一个32位整数强制转换为16位整数，在小端系统里，直接取前两个字节即可。所以，回顾之前我们讲解的，强制类型转换就是容易丢失数据。

## 5. 百度笔试题

请简述大端字节序和小端字节序的概念，设计一个小程序来判断当前机器的字节序。（10分）

### 代码1：指针类型转换法

```c
#include <stdio.h>
int check_sys()
{
    int i = 1;
    return (*(char *)&i);
}

int main()
{
    int ret = check_sys();
    if(ret == 1)
    {
        printf("小端\n");
    }
    else
    {
        printf("大端\n");
    }
    return 0;
}
```

### 代码2：联合体法

```c
//学习完联合体我们就能实现
int check_sys()
{
    union
    {
        int i;
        char c;
    }un;
    un.i = 1;
    return un.c;
}
```

### 本节总结

- 字节序是多字节数据在内存中的存储顺序。
- 小端字节序：低位字节存放在低地址（主流PC架构）。
- 大端字节序：高位字节存放在低地址（网络字节序）。
- 两种字节序各有优劣，是不同体系结构设计的产物。
- 可以通过指针类型转换或联合体来判断当前系统的字节序。

@博哥C语言制作